LEARNPULSE

Plataforma de Capacitaciones & Encuestas

ESPECIFICACIÓN TÉCNICA — v1.0

Abril 2026

CONFIDENCIAL — USO INTERNO DEL EQUIPO

Índice de Contenido

1. Visión del Producto
2. Stack Tecnológico
3. Arquitectura General del Sistema
4. Diseño de Base de Datos
5. ESPECIFICACIÓN BACKEND (NestJS)
   - 5.1 Estructura de Módulos
   - 5.2 Autenticación & Roles
   - 5.3 API REST — Contratos
   - 5.4 Redis — Uso exacto
   - 5.5 Manejo de Media
   - 5.6 BullMQ — Procesamiento Async
6. ESPECIFICACIÓN FRONTEND
   - 6.1 Stack y Estructura
   - 6.2 Módulos de UI
   - 6.3 Rutas y Páginas
   - 6.4 Estado Global
7. Infraestructura & DevOps
8. Testing
9. Roadmap MVP → Escalable

1. VISIÓN DEL PRODUCTO

LearnPulse es una plataforma SaaS multi-tenant para empresas que permite crear y gestionar Encuestas y Capacitaciones (trainings) de manera unificada, con soporte a media embebida, lógica condicional, analítica avanzada y emisión de certificados.

1.1 Tipos de Contenido

Tipo	Descripción	Certificado	Progreso
Encuesta	Recolecta opinión / feedback	No	Sí
Capacitación	Enseña + evalúa con preguntas	Sí	Sí

1.2 Decisiones Clave de Diseño

- Multi-tenant: cada empresa (Tenant) tiene sus propios usuarios, contenidos y datos aislados.
- Una "Encuesta" y una "Capacitación" comparten el mismo motor de contenido; la diferencia es configuración (aprobación, puntaje, certificado).
- El flujo puede ser secuencial obligatorio O libre — configurable por el creador.
- Branching (lógica condicional): si responde X → mostrar sección Y.
- Un usuario puede tener múltiples roles dentro de un tenant.

2. STACK TECNOLÓGICO

Capa	Tecnología	Justificación
Backend	NestJS + TypeScript	Modular, IoC, decoradores, perfecto para hexagonal
ORM	TypeORM / Prisma	Prisma recomendado: type-safe, migraciones limpias
Base datos principal	PostgreSQL 16	Relacional robusto, JSONB para contenido flexible
Cache / Sesiones	Redis 7	Sesiones JWT, cache, rate limit, progreso temporal
Colas async	BullMQ (sobre Redis)	Procesamiento de notificaciones, certificados PDF
Media storage	AWS S3	Videos/imágenes embebidas; PDFs de certificados
Auth	JWT + Refresh Token	Stateless, rotación de tokens segura
Frontend	Next.js 14 + TypeScript	SSR/SSG, App Router, SEO, performance
UI Library	shadcn/ui + Tailwind CSS	Componentes accesibles, personalización total
State	Zustand + React Query	Global ligero + server state con caché
Deploy	AWS ECS Fargate + RDS + ElastiCache	Serverless containers, managed DBs
CI/CD	GitHub Actions + Docker	Build → Test → Deploy automatizado

3. ARQUITECTURA GENERAL DEL SISTEMA

3.1 Vista de Alto Nivel

El sistema sigue una arquitectura de capas con separación clara entre el dominio de negocio y los detalles de infraestructura (Arquitectura Hexagonal / Ports & Adapters).

> *Cliente (Browser/Mobile) → CDN (CloudFront) → API Gateway → NestJS Backend → PostgreSQL / Redis / S3*

3.2 Principios Arquitectónicos Backend

- Domain Layer: Entidades, Value Objects, Interfaces de repositorios — SIN dependencias externas.
- Application Layer: Use Cases / Commands / Queries — orquesta el dominio.
- Infrastructure Layer: Repositorios concretos (TypeORM/Prisma), Redis, S3, BullMQ.
- Presentation Layer: Controllers NestJS, DTOs, Guards, Interceptors.

3.3 Flujo de Request

1. Request HTTP llega al Controller (Presentation).
2. Controller valida DTO y llama al Use Case (Application).
3. Use Case ejecuta lógica de negocio usando interfaces (Domain).
4. Infrastructure implementa las interfaces (DB, Redis, S3).
5. Response serializada y devuelta al cliente.

4. DISEÑO DE BASE DE DATOS (PostgreSQL)

4.1 Entidades Principales

El modelo se diseña en torno a Tenants (empresas), Users, Forms (encuestas/capacitaciones) y Responses.

TENANT

Campo	Tipo	Descripción
id	UUID PK	Identificador único
name	VARCHAR(120)	Nombre de la empresa
slug	VARCHAR(60) UNIQUE	URL-friendly identifier
plan	ENUM(free, pro, enterprise)	Plan de suscripción
settings	JSONB	Configuración personalizada del tenant

Campo	Tipo	Descripción
createdAt	TIMESTAMP	Fecha de creación

USER

Campo	Tipo	Descripción
id	UUID PK	
tenantId	UUID FK → Tenant	Tenant al que pertenece
email	VARCHAR UNIQUE per tenant	
passwordHash	VARCHAR	bcrypt
firstName / lastName	VARCHAR	
isActive	BOOLEAN	Soft disable
createdAt / updatedAt	TIMESTAMP	

ROLE & USER_ROLE (N:M)

Un usuario puede tener múltiples roles dentro de un tenant.

Rol	Descripción
SUPER_ADMIN	Administra la plataforma completa (interno LearnPulse)
TENANT_ADMIN	Administra todo dentro de su empresa
CREATOR	Crea y edita forms; puede responder para prueba
REVIEWER	Solo visualiza resultados y analítica
RESPONDENT	Solo resuelve forms asignados

FORM (encuesta o capacitación)

Campo	Tipo	Descripción
id	UUID PK	
tenantId	UUID FK	
title / description	VARCHAR / TEXT	
type	ENUM(survey, training)	
status	ENUM(draft, published, archived)	

Campo	Tipo	Descripción
config	JSONB	sequential, branching, passingScore, timeLimit, maxAttempts
version	INTEGER DEFAULT 1	Versionado de encuestas
parentFormId	UUID FK nullable	Para duplicar — apunta al original
createdBy	UUID FK → User	
createdAt / updatedAt	TIMESTAMP	

SECTION

Una Form se divide en Secciones (agrupadores lógicos), cada sección tiene Items.

Campo	Tipo	Descripción
id	UUID PK	
formId	UUID FK	
title	VARCHAR	
order	INTEGER	Orden de la sección
branchingRules	JSONB	Reglas: si questionId=X y value=Y → goto sectionId=Z

ITEM (corazón del contenido)

Campo	Tipo	Descripción
id	UUID PK	
sectionId	UUID FK	
type	ENUM(text, image, video, audio, pdf, question)	Tipo de ítem
order	INTEGER	Posición dentro de la sección
content	JSONB	Contenido variable según type (ver abajo)

Estructura del campo content (JSONB) según type:

type	Estructura JSONB de content
text	{ "body": "HTML string" }
image	{ "url": "https://...", "alt": "...", "caption": "..." }
video	{ "provider": "youtube|vimeo|s3", "embedId": "...", "url": "..." }

type	Estructura JSONB de content
audio	{ "url": "s3://...", "duration": 120 }
pdf	{ "url": "https://...", "title": "..." }
question	{ "questionType": "...", "text": "...", "options": [...], "correctAnswers": [...], "points": 1 }

QUESTION_TYPE (tipos de pregunta soportados)

Tipo	Descripción
single_choice	Opción única (radio buttons)
multiple_choice	Múltiple selección (checkboxes)
open_text	Respuesta abierta (textarea)
scale	Escala 1-N configurable
boolean	Sí / No
rating	Estrellas 1-5

ASSIGNMENT (asignación de forms a usuarios)

Campo	Tipo	Descripción
id	UUID PK	
formId	UUID FK	
userId	UUID FK	
assignedBy	UUID FK	Quién asignó
dueDate	TIMESTAMP nullable	Fecha límite
status	ENUM(pending, in_progress, completed, expired)	

RESPONSE_SESSION

Campo	Tipo	Descripción
id	UUID PK	
formId / userId	UUID FK	
attempt	INTEGER DEFAULT 1	Intento (si se permiten múltiples)

Campo	Tipo	Descripción
status	ENUM(in_progress, completed, abandoned)	
startedAt / completedAt	TIMESTAMP	
score	DECIMAL(5,2) nullable	Puntaje obtenido (capacitaciones)
passed	BOOLEAN nullable	Aprobado o no
progressSnapshot	JSONB	Cache de progreso (sección actual, respuestas parciales)

ITEM_RESPONSE

Campo	Tipo	Descripción
id	UUID PK	
sessionId	UUID FK	
itemId	UUID FK	
answer	JSONB	Respuesta flexible: { "selected": [...] } o { "text": "..." }
isCorrect	BOOLEAN nullable	Solo para preguntas con respuesta correcta
timeSpentMs	INTEGER	Tiempo en ms — analítica avanzada
answeredAt	TIMESTAMP	

CERTIFICATE

Campo	Tipo	Descripción
id	UUID PK	
sessionId	UUID FK UNIQUE	
userId / formId	UUID FK	
issuedAt	TIMESTAMP	
pdfUrl	VARCHAR	S3 URL del PDF generado
verificationCode	UUID UNIQUE	Código público de verificación

4.2 Índices Recomendados

- form.tenantId + form.status — consultas frecuentes de forms por tenant
- assignment.userId + assignment.status — mis encuestas pendientes

- response_session.formId + response_session.userId — histórico por usuario
- item.sectionId + item.order — carga ordenada de contenido
- certificate.verificationCode — verificación pública

5. ESPECIFICACIÓN BACKEND (NestJS)

5.1 Estructura de Módulos

Cada módulo sigue la estructura hexagonal interna: domain / application / infrastructure / presentation.

Módulo NestJS	Responsabilidad Principal
AuthModule	Login, registro, JWT, refresh tokens, guards
TenantModule	CRUD de tenants, settings, onboarding
UserModule	CRUD usuarios, asignación de roles
FormModule	CRUD de forms, versionado, duplicado
SectionModule	CRUD de secciones dentro de un form
ItemModule	CRUD de items (texto, media, preguntas)
AssignmentModule	Asignar forms a usuarios o grupos
ResponseModule	Iniciar sesión, guardar respuestas, finalizar
AnalyticsModule	Métricas, estadísticas, tiempos, abandono
CertificateModule	Generación de PDF, emisión, verificación pública
MediaModule	Presigned URLs para S3, validación de archivos
NotificationModule	Emails vía BullMQ (asignación, recordatorios)
HealthModule	Endpoint /health para AWS Load Balancer

Estructura de carpetas por módulo (ejemplo: FormModule)

src/modules/form/

- domain/ → entities/form.entity.ts, repositories/form.repository.interface.ts
- application/ → use-cases/create-form.usecase.ts, dtos/
- infrastructure/ → repositories/form.prisma.repository.ts
- presentation/ → form.controller.ts, form.module.ts

5.2 Autenticación & Roles

Aspecto	Implementación
Auth	JWT Access Token (15 min) + Refresh Token (7 días, rotación)
Guard	JwtAuthGuard global + RolesGuard por endpoint
Decoradores	@Roles(Role.CREATOR) @CurrentUser() @CurrentTenant()
Multi-tenant	tenantId extraído del JWT claim; toda query filtrada por tenantId
Refresh	POST /auth/refresh — nuevo access token; RT almacenado en Redis con TTL
Logout	Invalida RT en Redis (blacklist por jti)

Matriz de permisos por endpoint clave

Acción	RESPONDENT	REVIEWER	CREATOR	TENANT_ADMIN
Ver forms asignados	✓	✓	✓	✓
Responder form	✓	✗	✓(preview)	✓
Crear / editar form	✗	✗	✓	✓
Ver analítica	✗	✓	✓(propios)	✓
Gestionar usuarios	✗	✗	✗	✓
Emitir certificados	✗	✗	✗	✓

5.3 API REST — Contratos

> *Prefijo global: /api/v1 — todos los endpoints requieren header: Authorization: Bearer <token> salvo /auth/**

Auth

Método	Ruta	Descripción
POST	/auth/login	Login con email + password → { accessToken, refreshToken }
POST	/auth/register	Registro de usuario en un tenant
POST	/auth/refresh	Renovar access token con refreshToken
POST	/auth/logout	Invalida refresh token

Forms

Método	Ruta	Descripción
GET	/forms	Listar forms del tenant (paginado, filtros)

Método	Ruta	Descripción
POST	/forms	Crear form (CREATOR+)
GET	/forms/:id	Detalle de form con secciones e items
PATCH	/forms/:id	Actualizar form (crea nueva versión si publicado)
DELETE	/forms/:id	Archivar form (soft delete)
POST	/forms/:id/duplicate	Duplicar form
POST	/forms/:id/publish	Publicar form (cambia status)

Items

Método	Ruta	Descripción
GET	/forms/:formId/sections/:sId/items	Listar items de sección
POST	/forms/:formId/sections/:sId/items	Crear item (text/media/question)
PATCH	/items/:id	Actualizar item
DELETE	/items/:id	Eliminar item
PATCH	/sections/:sId/items/reorder	Reordenar items (array de ids + posiciones)

Responses (resolución de forms)

Método	Ruta	Descripción
POST	/responses/start	Iniciar sesión de respuesta → sessionId
POST	/responses/:sessionId/answer	Guardar respuesta de un item
POST	/responses/:sessionId/complete	Finalizar sesión → score, passed, certificado
GET	/responses/:sessionId/progress	Recuperar progreso guardado
GET	/responses/my	Historial de sesiones del usuario

Analytics

Método	Ruta	Descripción
GET	/analytics/forms/:id/summary	Conteo general, tasa de finalización, score promedio
GET	/analytics/forms/:id/questions	Distribución de respuestas por pregunta
GET	/analytics/forms/:id/users	Progreso y resultados por usuario
GET	/analytics/forms/:id/time	Tiempo promedio, abandono por sección

Media

Método	Ruta	Descripción
POST	/media/presigned-url	Generar presigned URL para subir a S3
POST	/media/confirm	Confirmar subida exitosa y registrar en DB
DELETE	/media/:id	Eliminar archivo de S3 y registro

Certificates

Método	Ruta	Descripción
GET	/certificates/my	Listar certificados del usuario autenticado
GET	/certificates/:id/download	Descargar PDF del certificado
GET	/certificates/verify/:code	Verificación pública por código (no requiere auth)

5.4 Redis — Uso Exacto

Uso	Key Pattern	TTL	Descripción
Refresh Tokens	rt:{userId}:{jti}	7 días	Set al login; del en logout/refresh
JWT Blacklist	bl:{jti}	15 min	Tokens revocados antes de expirar
Session Cache	sess:{sessionId}	48 h	Progreso temporal de sesión (sección actual, respuestas)
Form Cache	form:{formId}:v{version}	1 h	Form completo serializado; invalida en PATCH
Rate Limiting	rl:{ip}:{endpoint}	1 min	Máx 60 req/min por IP en auth
Analytics Counters	stats:{formId}:starts / completes	Persistente	Contadores atómicos INCR

5.5 Manejo de Media

Los videos se embeben (YouTube/Vimeo). Las imágenes, audios y PDFs se almacenan en AWS S3 mediante Presigned URLs para evitar pasar archivos por el backend.

6. Frontend solicita POST /media/presigned-url con { fileName, mimeType, size }.
7. Backend valida tipos permitidos (image/*, audio/*, application/pdf) y tamaño max.
8. Backend genera presigned URL de S3 con TTL de 5 min y devuelve { uploadUrl, fileKey }.
9. Frontend sube el archivo directamente a S3 usando la presigned URL (PUT).
10. Frontend notifica al backend POST /media/confirm con { fileKey } → backend registra el asset en DB.
11. Control de acceso: los objetos S3 son privados; el backend genera signed URLs de lectura con TTL.

5.6 BullMQ — Colas Async

Cola	Trigger	Job
email-queue	Asignación de form	Enviar email al respondent con link directo
email-queue	Recordatorio 24h antes de vencimiento	Email de recordatorio
certificate-queue	Session.completed + passed=true	Generar PDF certificado con Puppeteer → subir a S3
analytics-queue	Session.completed	Actualizar contadores y métricas en DB

6. ESPECIFICACIÓN FRONTEND (Next.js 14)

6.1 Stack y Estructura

Tecnología	Uso
Next.js 14 + App Router	Routing, SSR para SEO en landing, CSR para app
TypeScript	Tipos estrictos, DTOs compartidos con backend
Tailwind CSS + shadcn/ui	UI accesible, dark mode, theming por tenant
React Query (TanStack)	Cache de servidor, loading states, mutaciones
Zustand	Estado global: usuario, tenant, tema
React Hook Form + Zod	Validación de formularios con inferencia de tipos
Framer Motion	Animaciones suaves en el player de forms
Next-intl	Internacionalización (es / en)

Estructura de carpetas

- src/app/ → App Router (layouts, pages, loading, error)
- src/app/(auth)/ → Rutas de login, registro
- src/app/(dashboard)/ → Rutas autenticadas — creator/reviewer
- src/app/(player)/ → Rutas de resolución de forms — respondent
- src/app/(public)/ → Landing, verificación de certificados
- src/components/ → Componentes compartidos (UI atómica)
- src/features/ → Módulos por feature (forms, analytics, certificates...)
- src/lib/ → API client, utils, constants
- src/stores/ → Zustand stores
- src/types/ → Tipos TypeScript globales / DTOs

6.2 Módulos de UI

Módulo	Descripción	Roles que lo ven
AuthModule	Login, registro, forgot password	Todos
DashboardModule	Home con métricas rápidas y forms recientes	CREATOR, REVIEWER, ADMIN
FormBuilderModule	Editor drag-and-drop de secciones e items	CREATOR, ADMIN
FormPlayerModule	Interfaz de resolución secuencial/libre con progreso	RESPONDENT, CREATOR (preview)
AnalyticsModule	Dashboard de estadísticas, gráficos, exports	REVIEWER, CREATOR, ADMIN
AssignmentModule	Asignar forms a usuarios, fechas límite	CREATOR, ADMIN
UserManagementModule	CRUD usuarios, roles, invitaciones	ADMIN
CertificateModule	Ver y descargar mis certificados	RESPONDENT
TenantSettingsModule	Branding, logo, colores, configuración general	ADMIN
PublicVerifyPage	Verificar certificado por código (no requiere login)	Público

6.3 Rutas y Páginas

Ruta	Componente / Página	Descripción
/login	LoginPage	Formulario de acceso
/register	RegisterPage	Registro con tenant slug
/dashboard	DashboardHome	Resumen general
/forms	FormListPage	Tabla/grid de forms con filtros
/forms/new	FormBuilderPage	Crear nuevo form — builder vacío
/forms/:id/edit	FormBuilderPage	Editar form existente
/forms/:id/analytics	FormAnalyticsPage	Dashboard analítico del form
/forms/:id/assignments	AssignmentsPage	Gestión de asignaciones
/play/:formId	FormPlayerPage	Resolución del form (respondent)
/play/:formId/complete	CompletionPage	Pantalla de finalización + certificado
/my-forms	MyAssignedForms	Forms asignados al respondent
/certificates	CertificatesPage	Mis certificados obtenidos

Ruta	Componente / Página	Descripción
/verify/:code	PublicVerifyPage	Verificación pública (sin login)
/settings	TenantSettingsPage	Configuración del tenant
/users	UserManagementPage	Gestión de usuarios

6.4 Estado Global (Zustand)

Store	Estado que maneja
authStore	user, accessToken, tenant, isAuthenticated, login(), logout()
playerStore	sessionId, currentSectionIndex, answers (temp), timeElapsed
uiStore	sidebarOpen, theme, notifications

6.5 Form Builder — Consideraciones UX

- Drag & Drop de items dentro de una sección usando @dnd-kit/core.
- Panel izquierdo: lista de secciones navegables.
- Panel central: items ordenables de la sección activa.
- Panel derecho: propiedades del item seleccionado (tipo, opciones, branching).
- Auto-save cada 30 segundos via PATCH /forms/:id (debounced).
- Vista previa "Preview Mode" que abre FormPlayerPage en modo sandbox.
- Editor de branching visual: dropdowns "si pregunta X = valor Y → ir a sección Z".

7. INFRAESTRUCTURA & DEVOPS

7.1 Docker Compose (desarrollo local)

docker-compose.yml incluye los servicios mínimos para desarrollo:

- postgres:16 — Puerto 5432 — Volumen persistente
- redis:7-alpine — Puerto 6379
- backend — Build desde Dockerfile.dev — Puerto 3000 — Watch mode
- frontend — Build desde Dockerfile.dev — Puerto 3001 — Hot reload
- mailhog — Puerto 8025 — SMTP local para emails de desarrollo

> *Variables de entorno en .env (nunca en git). Usar .env.example como plantilla.*

7.2 Variables de Entorno (Backend)

Variable	Descripción
DATABASE_URL	PostgreSQL connection string

Variable	Descripción
REDIS_URL	Redis connection string
JWT_SECRET / JWT_REFRESH_SECRET	Secrets para firmar tokens
JWT_ACCESS_TTL / JWT_REFRESH_TTL	15m / 7d
AWS_REGION / AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Credenciales AWS
AWS_S3_BUCKET	Nombre del bucket
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS	Configuración email
APP_PORT	Puerto del servidor (default 3000)
CORS_ORIGINS	Orígenes permitidos

7.3 AWS — Producción

Servicio AWS	Uso
ECS Fargate	Containers del backend NestJS (auto-scaling)
RDS PostgreSQL	DB gestionada con backups automáticos
ElastiCache Redis	Redis gestionado (cluster mode off para MVP)
S3	Almacenamiento de imágenes, audios, PDFs, certificados
CloudFront	CDN para assets S3 y frontend Next.js
ECR	Registry de imágenes Docker
ALB	Application Load Balancer con health checks
Route 53	DNS y certificados SSL
Secrets Manager	Variables de entorno sensibles en producción

7.4 CI/CD con GitHub Actions

12. Push a main → trigger workflow.
13. Build Docker image (backend y frontend).
14. Run tests (unit + e2e).
15. Push imagen a ECR.
16. Deploy a ECS Fargate (rolling update).
17. Notificación de deploy a Slack/email.

8. ESTRATEGIA DE TESTING

Tipo	Herramienta	Qué cubrir	Cobertura objetivo
Unit	Jest	Use Cases, Domain entities, validaciones	80%
Integration	Jest + Supertest	Controllers + DB real (test container)	60%
E2E Backend	Supertest	Flujos completos: login → crear form → responder → certificado	Flujos críticos
E2E Frontend	Playwright	Flujos críticos: login, builder, player, dashboard	Flujos críticos
Unit Frontend	Vitest + Testing Library	Componentes, stores Zustand, hooks	70%

> *Usar test containers (Docker en CI) para PostgreSQL y Redis en tests de integración — nunca mockear la DB en tests de integración.*

9. ROADMAP MVP → ESCALABLE

Fase	Duración estimada	Entregables
Fase 1 — Core MVP	6–8 semanas	Auth multi-tenant completo CRUD de Forms con secciones e items básicos Form Player secuencial simple Guardar respuestas y finalizar sesión Deploy en AWS con Docker
Fase 2 — Capacitación	4–6 semanas	Preguntas con respuestas correctas y puntaje Lógica de aprobación configurable Generación de certificados PDF Branching / lógica condicional Progreso guardable (pausar y continuar)
Fase 3 — Analítica & UX	4 semanas	Dashboard analítico completo Tiempos por sección, tasa de abandono Form Builder drag-and-drop pulido Notificaciones y recordatorios por email Versionado de forms
Fase 4 — Escala	Continuo	Internacionalización (i18n) Branding personalizado por tenant API pública para integraciones Mobile app (React Native)

Fase	Duración estimada	Entregables
		Auto-scaling ECS